

SIMATS ENGINEERING

ASSESSMENT TOOL 2 - DEBUGGING & OPTIMISATION TASK SOLUTIONS

QUESTION 1: Signed Integer Addition & Overflow Detection in 2's Complement

1. Binary Computation & Issue Analysis

- **Operation:** Adding +120 and +50 in an 8-bit signed integer representation.

- **+120 in 8-bit Binary:** 0111 1000₂

- **+50 in 8-bit Binary:** 0011 0010₂

- **Binary Addition:** 0111 1000 (+120)

+ 0011 0010 (+50)

1010 1010₂ (-86 in 2's complement)

- **Identified Issue:** Adding two positive numbers (+120 and +50) produced a negative result (-86). This indicates an Arithmetic Overflow.

2. Root Cause

- **Range Exceeded:** An 8-bit signed integer using 2's complement representation has a valid range of -128 to +127 [-2^7 to $2^7 - 1$]. The mathematical sum $120 + 50 = 170$ exceeds the maximum positive limit (+127). The value wrapped around into the negative region due to bit truncation.

3. Debugging & Proposed Solution

- **Overflow Detection Logic:** In signed 2's complement addition, overflow occurs if and only if:

- Two positive numbers yield a negative result, OR
- Two negative numbers yield a positive result.

In hardware terms, Overflow (V) = $C_{in} \oplus C_{out}$ (where C_{in} is carry into the MSB/sign bit position, and C_{out} is carry out of the MSB position).

- **Handling & Prevention:** 1. Hardware Flag Monitoring: Check the ALU Overflow Flag (V) after addition. If $V = 1$, trigger an arithmetic exception.

2. Bit Extension: Promote variables to a wider data type (e.g., 16-bit signed integers) prior to computation, extending the dynamic range [-32,768 to +32,767].

QUESTION 2: Ripple Carry Adder (RCA) Bottleneck & Carry Look-Ahead Adder (CLA) Optimization

1. Performance Bottleneck Analysis

- **Propagation Delay Mechanics:** In an n-bit Ripple Carry Adder (RCA), each Full Adder stage (i) cannot compute its final Sum (S_i) and Carry Out (C_{i+1}) until the Carry In (C_i) propagates from the preceding stage (i-1).
- **Time Complexity:** The total worst-case propagation delay scales linearly with bit width: $T_{\text{delay}} = O(n) \times t_{\text{gate}}$. For a 32-bit or 64-bit real-time processor, this sequential ripple propagation causes significant latency, stalling the instruction execution pipeline.

2. Carry Look-Ahead Adder (CLA) Optimization

- **Parallel Carry Generation:** CLA eliminates serial carry dependency by pre-computing carry signals in parallel using bit-wise Generate (G) and Propagate (P) logic functions:
- **Generate Function (G_i):** $G_i = A_i \cdot B_i$ (Generates a carry regardless of carry-in)
- **Propagate Function (P_i):** $P_i = A_i \oplus B_i$ (Propagates incoming carry)
- **Look-Ahead Equations:**
$$C_1 = G_0 + P_0 \cdot C_0$$
$$C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$$
$$C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$$
$$C_4 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

3. How Delay is Reduced

- **Constant Logic Depth:** All carry signals are derived directly from the inputs (A_i, B_i) and initial carry-in (C_0) simultaneously within 2-3 logic gate levels.
- **Latency Improvement:** Execution delay is reduced from $O(n)$ to constant/logarithmic time complexity $O(1)$ or $O(\log n)$, eliminating real-time processing bottlenecks.

QUESTION 3: Debugging Step-by-Step Execution of Booth's Multiplication Algorithm

1. Source of Error Identification

- **Common Failure Modes:** Incorrect negative multiplication results in Booth's algorithm stem from:
 1. Improper Arithmetic Right Shift (ARS) failing to preserve the sign bit (MSB).
 2. Failure to sign-extend operands to full register bit-width before starting execution.
 3. Misinterpreting the bit pair (Q_0, Q_{-1}) transitions.

2. Bit-Pair Rules & Arithmetic Operations

- **Inspection Pairs (Q_0, Q_{-1}):** • 0 0 $\rightarrow$ No operation, Arithmetic Right Shift (ARS) [A, Q, Q_{-1}]

- 0 1 $\rightarrow A = A + M$, then ARS $[A, Q, Q_{-1}]$
- 1 0 $\rightarrow A = A - M$ (or $A = A + M' + 1$), then ARS $[A, Q, Q_{-1}]$
- 1 1 $\rightarrow$ No operation, Arithmetic Right Shift (ARS) $[A, Q, Q_{-1}]$

3. Debugging Verification & Corrections

- **Arithmetic Right Shift (ARS) Verification:** Unlike logical right shift, ARS must replicate the Most Significant Bit (MSB):

Example: If $A = 10110010$, $ARS(A) = 11011001$ (MSB '1' is preserved). Replacing ARS with Logical Shift clears the sign bit, corrupting negative numbers.

- **Sign Extension Protocol:** Ensure multiplicand (M) and accumulator (A) are sign-extended to double the operand size during 2's complement subtraction ($A - M$) to avoid bit truncation errors.

QUESTION 4: Restoring vs. Non-Restoring Division Algorithm Optimization

1. Inefficiency in Restoring Division

- **Redundant Restoring Steps:** In Restoring Division, for each bit iteration:

1. Shift left register pair (A, Q).
2. Perform trial subtraction: $A = A - M$.
3. Test sign of A:
 - If $A \geq 0$ (positive), set $Q_0 = 1$.
 - If $A < 0$ (negative), set $Q_0 = 0$ AND restore original value: $A = A + M$.

Performance Impact: When subtraction fails, two arithmetic operations (subtract then add) occur in a single cycle step, doubling total operation count.

2. Non-Restoring Division Optimization

- **Eliminating Restoration Steps:** Non-Restoring Division avoids restoring addition on failed trial subtractions by combining steps into the next cycle:

- **Algorithm Rules:**
 - If previous $A \geq 0$: Shift Left (A, Q), then Subtract $M \rightarrow A = 2A - M$
 - If previous $A < 0$: Shift Left (A, Q), then Add $M \rightarrow A = 2A + M$
 - Set Q_0 based on sign of resulting A ($Q_0 = 1$ if $A \geq 0$; $Q_0 = 0$ if $A < 0$).
 - Final Correction: If remainder A is negative at the end of all steps, execute a single final restoration: $A = A + M$.

3. Performance Improvement

- **Execution Time Reduction:** Requires exactly 1 arithmetic operation (add or subtract) per iteration bit, achieving ~50% reduction in ALU cycle operations compared to Restoring Division.

QUESTION 5: IEEE 754 Single-Precision Floating-Point Precision Loss & Optimizations

1. Issue Analysis: Exponent Alignment & Precision Loss

- **Alignment Mechanism:** In IEEE 754 floating-point addition, exponents must be matched before significand addition. The significand of the smaller number is shifted right by $\Delta E = |E_1 - E_2|$ positions.
- **Catastrophic Loss (Underflow):** In 32-bit single precision (1 sign bit, 8 exponent bits, 23 explicit mantissa bits):

If $\Delta E > 24$, shifting the small number right causes all its significand bits to shift past the LSB limit into zero, resulting in $A + B = A$. The small value is completely ignored.

2. Optimization & Debugging Techniques

- **Technique 1: Extended Precision (IEEE 754 Double Precision):** Switch from single precision (32-bit, 23 mantissa bits) to double precision (64-bit, 52 mantissa bits). Increases dynamic range and mantissa capacity, handling magnitude differences up to 2^{52} .
- **Technique 2: Kahan Summation Algorithm (Algorithmic Adjustment):** Maintains a running compensation variable (c) to track lost low-order bits during summation, restoring accumulated lost precision in subsequent additions.
- **Technique 3: Guard, Round, and Sticky Bits (Hardware ALU Rounding):** Utilize G, R, S rounding bits beyond the 23rd bit position during alignment and normalization (Rounding to Nearest Even) to preserve fractional accuracy.